CSC 4553

Fall 2026 - Day 3

Admin

- Programming assignment 1 is out
- Lab 2 due Friday, 18:00 (includes use of tools that will be useful for PA1)

Boot chain

Questions to consider

- What is the chain of events that takes place before you are able to run a process?
- Where do the different parts of the chain reside? How are they called?

Boot is like a relay race

- A sequence of programs
- Each stage:
  - → Has slightly more power
  - → Loads the next stage
  - → Jumps to it

Power-On

- CPU resets
- Instruction pointer set to a fixed address
- Execution begins in firmware (ROM)
- The CPU does not know what an OS is

Firmware (BIOS / UEFI)

- Initializes RAM
- Initializes minimal hardware
- Finds one program to run (bootloader)
- Loads it into memory
- Jumps to it

Bootloader’s job (GRUB Example)

- Understand filesystems
- Load the kernel image
- Load the initramfs
- Jump to the kernel entry point
- Provides kernel command line args

```
root=/dev/sda1
init=/sbin/init
console=ttyS0
```

What the bootloader does **not** do

- Does not schedule processes
- Does not manage memory long-term
- Does not run the OS

Kernel

Think of the kernel in phases

1. Self-setup
2. Hardware abstraction
3. Userspace handoff

Kernel self-setup

- Sets up page tables
- Enables virtual memory
- Initializes scheduler
- Sets up interrupts

The kernel cannot rely on **anything** yet: it is building the world.

Kernel hardware abstractions

This is where hardware becomes an abstraction.

- Detects hardware
- Loads drivers
- Turns devices into files:
  - → `/dev/sda`
  - → `/dev/tty`
- Creates kernel threads

Userspace handoff

- Mounts the root filesystem
- Chooses one program to run
- Executes it with `execve()`

```
1  execve("/sbin/init", ...);
```

Init (PID 1)

- First userspace process
- Always PID 1
- Parent of all other processes

If PID 1 exits?

- The kernel **panics** or shuts down

What isn’t clear?

Comments? Thoughts?

Kernel basics

Questions to consider

- What are the different kernel paradigms?
- Why would you choose one over the other?

Paradigms

- The OS offers a number of services. What should go in the kernel?
  - → IPC
  - → VFS
  - → File system
  - → Scheduler
  - → Virtual Memory
  - → Device drivers

Monolithic kernels

- Oldest, very common design (Linux, Windows 9x, BSDs)
- Single piece of code in memory
- Limited **information hiding**

  → One part of the kernel can directly access data and functions of other parts

Monolithic kernels (cont'd)

- Q: What happens when you need something new supported (new hardware device, new system call, new filesystem)?
- **Modules** (loadable kernel modules - LKMs) allow for flexibility, customization, support
  - → Pros?
    - Memory savings
    - Flexibility
    - Minimal downtime
  - → Cons?
    - (minor) fragmentation: the base kernel can be loaded contiguously in memory
    - Security (https://github.com/m0nad/Diamorphine **DO NOT USE THIS, IT'S SIMPLY TO SHOW YOU**)

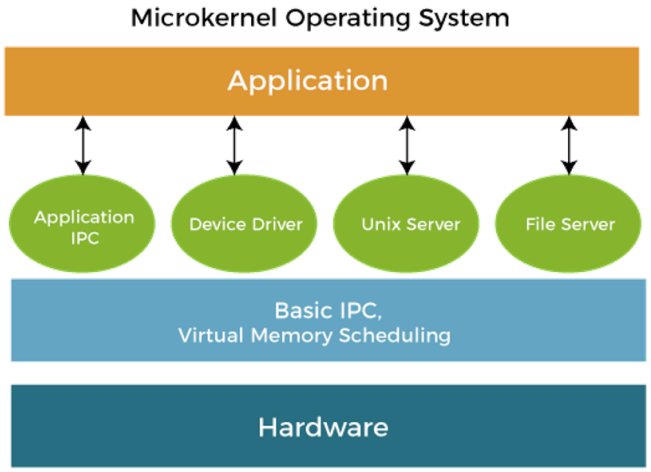
Layered kernels

- Dijkstra created the THE OS in the 60s, introducing the concept Each inner layer is more privileged; required a trap to move down layers
- Hardware-enforcement possible
- Intel **announced** in May 2024 that the new architectures will only have rings 0 and 3

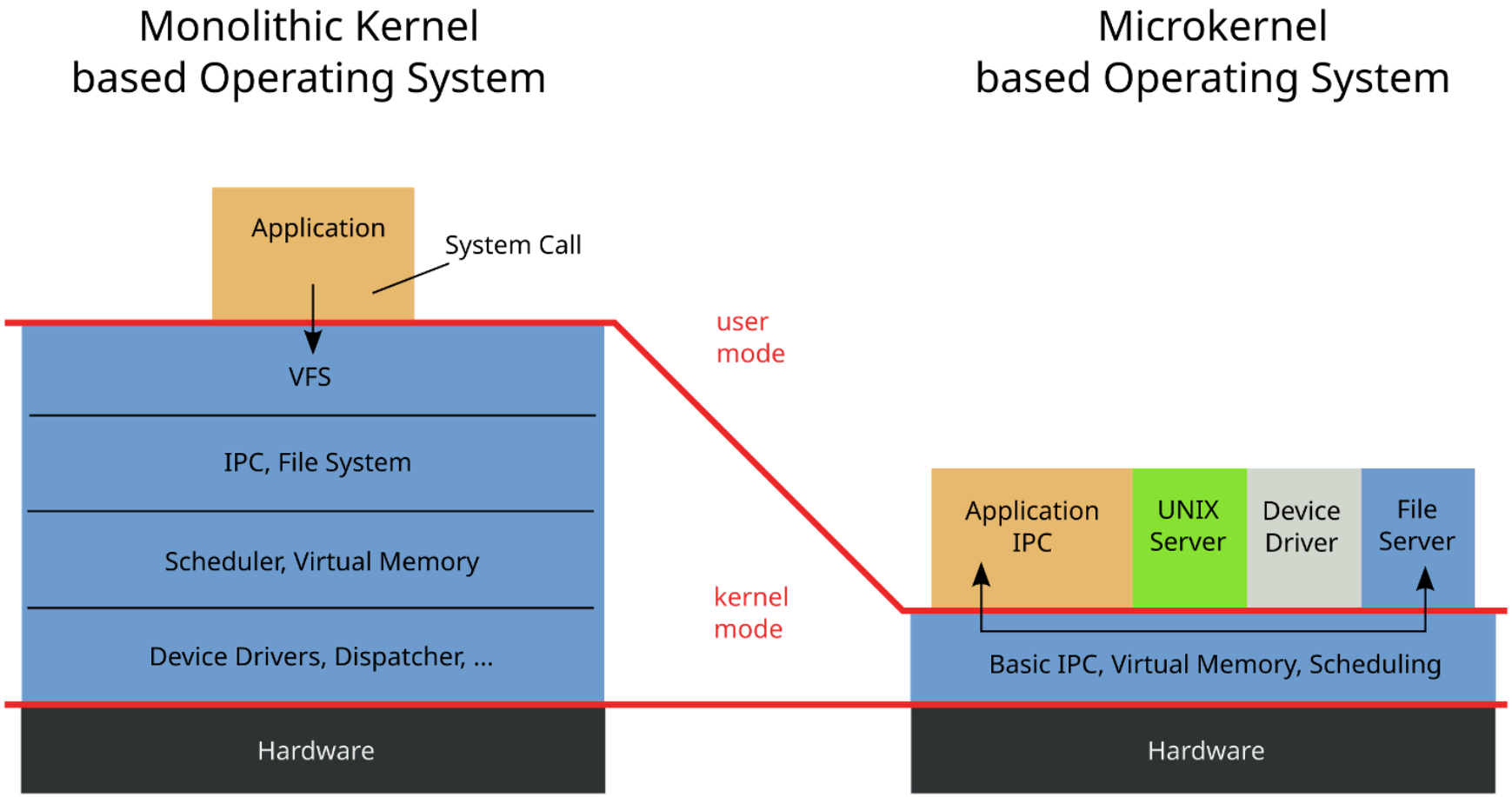
Microkernels

- Popular research area long ago, didn't win (although people remain interested in the principles)
- All non-essential components removed from the kernel, for modularization, extensibility, isolation, security, and ease of management
- A collection of OS services running in user space
- Downsides?

  → **Heavy communication costs** through message passing (marshaled through the kernel)



Monolithic vs Microkernel



Hybrid kernels

- Combine a monolithic core with microkernel ideas
- Small kernel handles critical services (scheduling, IPC, low-level memory)
- Other services (file systems, drivers) can run in kernel *or* user space
- Examples: macOS (XNU), Windows NT
  - → XNU: Mach microkernel + BSD monolithic layer
  - → Windows NT: microkernel-inspired, but most services run in kernel mode for performance
- Pragmatic compromise: **microkernel isolation where it matters, monolithic speed where it doesn't**

What isn’t clear?

Comments? Thoughts?